Labsheet 7 - Computer vision

Name –Vishavjeet Baraily

3rd year section B

roll no. 68

```
import cv2
import numpy as np
cap = cv2.VideoCapture("video.mp4")
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
# Detect feature points
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
mask = np.zeros_like(old_frame)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # Lucas-Kanade Sparse Optical Flow
    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None
    )

    if p1 is not None:
        good_new = p1[status == 1]
```

```
        good_old = p0[status == 1]
        for new, old in zip(good_new, good_old):
            x1, y1 = new.ravel()
            x2, y2 = old.ravel()
            mask = cv2.line(
                mask,
                (int(x1), int(y1)),
                (int(x2), int(y2)),
                (0, 255, 0),
                2
            )
            frame = cv2.circle(
                frame,
                (int(x1), int(y1)),
                4,
                (0, 0, 255),
                -1
            )
    output = cv2.add(frame, mask)
    cv2.imshow("Lucas-Kanade Optical Flow", output)
    # Update previous frame and points
    old_gray = frame_gray.copy()
    p0 = good_new.reshape(-1, 1, 2)
    if cv2.waitKey(30) & 0xFF == 27:
        break
cap.release()
cv2.destroyAllWindows()
```

Farneback Dense Optical Flow

```
import cv2
import numpy as np
cap = cv2.VideoCapture("video.mp4")
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
while True:
    ret, frame = cap.read()
```

```
    if not ret:
        break
    new_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # Farneback Dense Optical Flow
    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        new_gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    # Magnitude and direction
    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )
    # HSV visualization
    hsv = np.zeros_like(frame)
    hsv[..., 0] = angle * 180 / np.pi / 2
    hsv[..., 1] = 255
    hsv[..., 2] = cv2.normalize(
        magnitude, None, 0, 255, cv2.NORM_MINMAX
    )
    output = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
    cv2.imshow("Farneback Dense Optical Flow", output)
    old_gray = new_gray.copy()
    if cv2.waitKey(30) & 0xFF == 27:
        break
cap.release()
```

cv2.destroyAllWindows()

Questions and Answers

1. What is Optical Flow? How is it used in computer vision?

Optical Flow is the apparent motion of pixels or objects between consecutive video frames. It represents motion using vectors showing the direction and magnitude of movement.

It is used for:

- **Object tracking**
- **Motion detection**
- **Video stabilization**
- **Autonomous driving**
- **Action recognition**

---

2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

Lucas-Kanade is a Sparse Optical Flow method. It first selects important feature points such as corners and then tracks their movement between consecutive frames.

It assumes that nearby pixels have similar motion. Using image gradients and equations, it calculates the displacement of each selected feature point.

---

3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Dense Optical Flow calculates motion for almost every pixel in an image.

Sparse Optical Flow calculates motion only for selected feature points.

- **Sparse → fewer points, faster computation.**
- **Dense → almost every pixel, more detailed motion information.**

---

4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Lucas-Kanade	Farneback
Sparse optical flow	Dense optical flow
Tracks selected feature points	Estimates motion for almost every pixel
Faster	More computationally expensive
Good for object tracking	Good for complete motion analysis
Uses local image information	Uses polynomial approximation

---

5. What assumptions are made while computing optical flow?

The main assumptions are:

1. **Brightness constancy: Pixel brightness remains approximately constant between frames.**
2. **Small motion: Object displacement between consecutive frames is relatively small.**
3. **Spatial coherence: Neighboring pixels generally have similar motion.**

---

6. What factors can affect the accuracy of optical flow estimation?

Accuracy can be affected by:

- **Fast object movement**
- **Sudden lighting changes**
- **Motion blur**
- **Camera movement**
- **Occlusion**
- **Low-texture or uniform regions**
- **Large displacement between frames**
- **Noise in the video**

---

7. Mention five real-world applications of optical flow.

1. **Autonomous vehicles – motion and obstacle detection.**
2. **Video surveillance – detecting moving objects.**
3. **Robotics – navigation and movement estimation.**
4. **Sports analytics – tracking players and objects.**
5. **Gesture recognition – detecting hand and body movements.**

---

8. Why are grayscale images generally used for optical flow computation?

Grayscale images contain intensity information without separate color channels. Optical flow mainly depends on changes in intensity between frames.

Using grayscale also:

- **Reduces computational complexity.**
- **Makes gradient calculation simpler.**
- **Speeds up processing.**

---

9. What are the limitations of optical flow algorithms in real-world environments?

Some limitations are:

- **Difficulty with very fast motion.**
- **Sensitive to lighting changes.**

- **Problems with occlusion.**
- **Less reliable in textureless areas.**
- **Camera movement can create unwanted motion.**
- **Dense methods can require high computational power.**

---

10. How does optical flow contribute to autonomous driving, video surveillance, and action recognition?

Autonomous Driving: Helps estimate vehicle, pedestrian, and road motion.

Video Surveillance: Helps detect and track moving objects or unusual movements.

Action Recognition: Helps identify movements such as walking, running, waving, or other human actions.

Thus, optical flow provides important motion information for understanding dynamic scenes.